Branch Coverage Report

Project	Online Exam Portal (Software Engineering mini project)
Team	Rehan S Bawakhan (CS820), Yeshaswinie D (CS619), Yashwantha R (CS614), Yashaswini Singh (CS612)
Build	`main` at commit `2f93f79`
Test run	11-11-2026 21:00 IST
Generated by	`docs/reports/generate_reports.py` (do not edit by hand; run the script again)
References	SRS v1.0, SAD v1.0, Software Test Plan v1.0, Test Case Document

1. Summary

Metric	Value
Branches (decision outcomes)	506
Covered	455
Missed	51
Partially covered decisions	49
Branch coverage	**89.9%**
Target (team)	≥ 75%
Result	**Met**

2. Method

coverage.py runs in branch mode (`branch = True` in `.coveragerc`). Every `if`, `elif`, loop, `try`/`except` and boolean short-circuit with two possible exits counts as two branches; a branch is covered when the test suite takes that exit at least once. A *partial* decision is one where only one of its exits was taken. Same code and same run as the Code Coverage Report.

3. Branch coverage by module

App	SAD module	Branches	Covered	Missed	Partial decisions	Branch coverage	≥ 75%?
`accounts`	Auth Service, Admin Service, Audit & Logging	116	111	5	5	95.7%	Met
`config`	Platform (settings, TLS, health check)	12	11	1	1	91.7%	Met
`courses`	Admin Service (courses)	16	14	2	2	87.5%	Met
`evaluation`	Evaluation Service	30	28	2	2	93.3%	Met
`exam_delivery`	Exam Delivery Service, diagnostics	100	79	21	19	79.0%	Met
`exam_management`	Exam Scheduling Service	62	57	5	5	91.9%	Met
`question_bank`	Question Bank Service	98	84	14	14	85.7%	Met
`reports`	Reporting Service	46	45	1	1	97.8%	Met
`web`	Web UI	26	26	0	0	100.0%	Met

4. Branch coverage by file

File	Branches	Covered	Partial	Branch coverage
`accounts/audit.py`	6	6	0	100.0%
`accounts/audit_models.py`	4	4	0	100.0%
`accounts/audit_views.py`	20	19	1	95.0%
`accounts/authentication.py`	8	8	0	100.0%
`accounts/backends.py`	10	8	2	80.0%
`accounts/lockout.py`	6	5	1	83.3%
`accounts/management/commands/archive_audit_logs.py`	6	5	1	83.3%
`accounts/models.py`	4	4	0	100.0%
`accounts/serializers.py`	30	30	0	100.0%
`accounts/views.py`	22	22	0	100.0%
`config/health.py`	2	1	1	50.0%

File	Branches	Covered	Partial	Branch coverage
`config/settings.py`	10	10	0	100.0%
`courses/serializers.py`	16	14	2	87.5%
`evaluation/scorecard.py`	6	6	0	100.0%
`evaluation/services.py`	4	4	0	100.0%
`evaluation/subjective.py`	18	16	2	88.9%
`evaluation/views.py`	2	2	0	100.0%
`exam_delivery/diagnostics.py`	6	5	1	83.3%
`exam_delivery/models.py`	4	3	1	75.0%
`exam_delivery/services.py`	90	71	17	78.9%
`exam_management/serializers.py`	14	13	1	92.9%
`exam_management/services.py`	44	40	4	90.9%
`exam_management/views.py`	4	4	0	100.0%
`question_bank/grading.py`	16	14	2	87.5%
`question_bank/import_utils.py`	40	32	8	80.0%
`question_bank/serializers.py`	24	20	4	83.3%
`question_bank/views.py`	18	18	0	100.0%
`reports/exports.py`	30	30	0	100.0%
`reports/services.py`	14	13	1	92.9%
`reports/views.py`	2	2	0	100.0%
`web/services.py`	10	10	0	100.0%
`web/views.py`	16	16	0	100.0%

Files with no decisions (for example `urls.py`, `apps.py`) are left out.

5. Missed branches

Each entry is `line → destination`; a destination of `exit` means leaving the function. Line numbers refer to the build above.

File	Missed	Branches
`accounts/audit_views.py`	1	53→55
`accounts/backends.py`	2	34→35, 36→37
`accounts/lockout.py`	1	21→22
`accounts/management/commands/archive_audit_logs.py`	1	80→81
`config/health.py`	1	25→26
`courses/serializers.py`	2	34→35, 48→49
`evaluation/subjective.py`	2	81→83, 152→153
`exam_delivery/diagnostics.py`	1	44→45
`exam_delivery/models.py`	1	83→84
`exam_delivery/services.py`	19	43→44, 63→64, 132→144, 261→262, 287→288, 299→300, 310→311, 319→322, 322→323, 322→326, 330→309, 400→401, 407→409, 409→410, 428→429, 487→488, 491→492, 494→495, 555→556
`exam_management/serializers.py`	1	43→44
`exam_management/services.py`	4	151→152, 154→155, 157→158, 190→191
`question_bank/grading.py`	2	35→36, 38→39
`question_bank/import_utils.py`	8	31→32, 100→101, 173→177, 181→182, 184→185, 202→203, 230→231, 243→244
`question_bank/serializers.py`	4	24→25, 50→54, 85→86, 95→96
`reports/services.py`	1	81→82

6. Analysis

The decisions that matter most for exam integrity are fully covered: server-side timer and deadline checks, single-attempt enforcement, answer-key filtering, role checks, lockout counting, and score finalisation (see `exam_delivery/services.py`, `exam_management/services.py`, `accounts/permissions.py`, `accounts/lockout.py`, `evaluation/services.py` above). The missed branches are mostly the `else` side of defensive checks (a value that is already validated upstream, an unexpected database error, a malformed spreadsheet cell type) and production-only configuration paths.